

Metro Ticket Generation System

Phase 2 — Backend Development & Configurations | ServiceNow

Phase Purpose	Build the ServiceNow backend foundation required to store, validate and manage metro data.
Primary Focus	Data architecture, table creation, field creation, relationships, configuration and backend readiness.
Starting Point	Phase 1 approved requirements, scope, stakeholders, business rules and MVP.
Expected Outcome	A structured ServiceNow data layer ready for business logic, automation, UI and ticket-generation features.

1. Phase 2 Overview

Phase 2 converts the requirements defined in Phase 1 into a working ServiceNow backend structure. The key idea is to design the data correctly first, then create tables and fields that support the metro ticket lifecycle.

Requirement → Data Architecture → Tables → Fields → Relationships → Validation/Configuration → Backend Testing → Ready for Logic

2. Brainstorming Before Configuration

- What data must the system remember for a station, route, fare, booking and ticket?
- Which data should be stored once and referenced rather than duplicated?
- Which fields are mandatory, unique, calculated, choice-based or reference fields?
- Which tables need relationships with one another?
- Who should be able to create, update, or only read each type of record?
- Which values should be configurable instead of hardcoded in scripts?
- What validations should happen at field/table level before business logic is introduced?

Recommended backend principle: store master data once, reference it where required, and keep transactional records separate from configuration data.

3. Data Architecture

The data architecture defines the logical structure of the Metro Ticket Generation System. It separates master/configuration data from transactional data so the application remains maintainable.

Table / Entity	Category	Purpose
Metro Station Details	Master	Stores available metro stations and their active/inactive state.
Metro Route	Master	Represents route or station connectivity information.
Metro Fare	Configuration	Stores fare rules/amounts used by the fare calculation process.
Metro Booking	Transaction	Stores a passenger's booking/journey request.
Metro Ticket	Transaction	Stores the generated ticket and its lifecycle/status.
Passenger	Reference / Master	Stores or references passenger information where required.

4. Conceptual Relationship Model

Metro Station provides selectable locations → Metro Booking references source and destination → Metro Fare supplies pricing → Metro Ticket is generated from a successful booking.

Relationship	Meaning
Station → Booking	A booking has one source station and one destination station; both should reference valid station records.
Fare → Booking	A booking uses the fare rule applicable to its journey.
Booking → Ticket	A successful booking produces the corresponding ticket record.
Passenger → Booking	A passenger can have multiple bookings over time.

5. Data Design Decisions

- Reference fields: Prefer references to master records for stations and other reusable entities.
- Choice fields: Use choices for controlled values such as ticket status and ticket type.
- Unique identifiers: Ticket number must be unique; station code can also be unique if introduced.
- Audit fields: Created/updated information should be retained using ServiceNow record auditing conventions.
- Active flag: Master records should support activation/deactivation without deleting historical references.

6. Data Architecture Quality Checklist

- No unnecessary duplicate master data
- Naming convention defined
- Mandatory fields identified
- Reference relationships identified
- Choice values documented
- Unique fields identified
- Active/inactive behavior defined
- Security ownership considered

7. Creation of Table — Metro Station Details

The first backend configuration activity is creating the table that stores metro station master data. In ServiceNow, fields are created at the table level, so the correct table must exist before the required fields can be added.

Activity 1 — Create the Table

Property	Value
Label	Metro Station's Details
Name	u_metro_station_details
Purpose	Store metro station master information used by booking and validation logic.
Application	Use the project's scoped application where applicable.

8. Step-by-Step ServiceNow Procedure

- Open the Application Navigator on the left side of ServiceNow.
- Search for Tables.
- Under System Definition, open Tables.
- Select New to create a table.
- Enter the table label as Metro Station's Details.
- Confirm the generated/custom table name as u_metro_station_details (subject to application/scope naming rules).
- Review the application and table settings before saving.
- Click Submit to create the table.
- Open the newly created table and verify that the table record exists.

9. Why Table Creation Comes First

A ServiceNow table is the backend container for records. Once the table exists, fields become columns in that table and can be exposed through forms, lists, scripts, flows and other application components.

Table = container | Field = column | Record = one stored station

10. Initial Table Validation

- Table label is correct and understandable.
- Table name follows the project's naming convention.
- Table belongs to the intended application/scope.
- The table is visible under System Definition → Tables.
- No duplicate table was created accidentally.
- The table is ready for field creation.

11. Creation of Fields — Metro Station Details

In ServiceNow, fields are created at the table level. After selecting the Metro Station's Details table, the required columns can be added from the Columns related list. Fields may also be configured through form-related tools depending on the development approach.

Activity 2 — Create the Station Name Field

Property	Value
Field Label	Station Name
Column / Name	u_station_name
Suggested Type	String
Purpose	Stores the human-readable name of the metro station.
Example	Ameerpet

12. Step-by-Step Field Creation

- In the Application Navigator, search for Tables.
- Open System Definition → Tables.
- Search for and select Metro Station's Details.
- Open the table record and locate the Columns tab/related list.
- Click Insert New Row to create a field.
- Enter the field label Station Name.
- Use the field/column name u_station_name according to the project naming convention.
- Select an appropriate data type such as String.
- Save/update the table configuration.
- Verify that the Station Name field appears in the table's column list and can be placed on the form.

13. Field Design Considerations

Consideration	Recommendation
Mandatory	Station Name should normally be mandatory because a station without a name is not useful.
Length	Choose a reasonable maximum length based on the organization's naming standard.
Uniqueness	Consider preventing duplicate station names or, preferably, introduce a unique station code.
Active state	Use an active/inactive mechanism so historical data is preserved when a station is retired.
Naming	Use consistent labels and scoped column names throughout the application.

14. Important Note

The provided Phase 2 material specifically defines creation of one field: Station Name (u_station_name). Additional fields should be introduced only after confirming the project's approved data model rather than creating unnecessary columns.

15. Backend Configuration Plan

After the first table and field are created, the same controlled approach can be applied to the remaining backend entities required by the approved architecture.

Component	Initial Configuration Goal
Metro Station Details	Create table and Station Name field; establish station master foundation.
Metro Route	Define route/station relationship structure if required by the approved fare/route model.
Metro Fare	Store configurable fare rules rather than hardcoding amounts in business logic.
Metro Booking	Store passenger journey inputs, fare result, booking state and references.
Metro Ticket	Store unique ticket number, booking reference, validity and status.

16. Configuration Standards

- Naming: Use clear labels and consistent scoped names such as u__ where applicable.
- Data types: Select the smallest suitable type that represents the business value correctly.
- References: Use reference fields when a field represents another ServiceNow record.
- Choices: Use controlled choice values for status/type fields.
- Validation: Mark required fields and add appropriate validation before downstream logic.
- Reusability: Keep station and fare master data independent from individual ticket transactions.
- Security: Decide which roles can create/read/update/delete each backend table.
- Auditability: Avoid destructive deletion of master data when historical transactions may depend on it.

17. Brainstorming — Backend Questions

Question	Decision Needed
Do we need a station code?	Recommended for unique identification and integrations.
Do we need station order/distance?	Required if route/distance-based fare is used.
Should fare be hardcoded?	No; prefer configurable fare records/rules.
Should source/destination be plain text?	Prefer references to Metro Station records.
Should booking and ticket be one table?	Separate them when ticket lifecycle and booking lifecycle need independent tracking.
Who can change fare data?	Restrict to an authorized admin/operator role.
Can inactive stations be selected?	Normally no; historical records should remain intact.

18. Backend Development Flow

Create Table → Define Fields → Define Relationships → Configure Form/List → Add Validation → Add Security → Test Records → Proceed to Business Logic

19. Testing & Verification for Phase 2

Phase 2 testing verifies that the backend structure works correctly before scripts, flows and ticket-generation automation are built.

Test Case	Steps	Expected Result
Table creation	Search Tables for Metro Station's Details.	Correct table exists with intended label/name.
Field creation	Open table → Columns.	Station Name / u_station_name appears.
Record creation	Create a station record with a valid name.	Station record saves successfully.
Mandatory validation	Leave Station Name empty if configured mandatory.	Record is rejected with validation message.
Duplicate check	Create two identical stations if uniqueness is configured.	Second duplicate is rejected.
Inactive station	Deactivate a station and attempt booking selection later.	Inactive station should not be selectable if that rule is implemented.
Reference integrity	Use a station as source/destination in booking.	Booking stores a valid station reference.
Access control	Test with passenger/operator/admin roles.	Each role receives only intended access.

20. Phase 2 Risks & Mitigation

Risk	Impact	Mitigation
Poor data model	High	Review relationships and requirements before creating dependent tables.
Incorrect field type	Medium	Confirm business meaning and expected values before configuration.
Duplicate master data	Medium	Use unique identifiers/validation where appropriate.
Over-customization	Medium	Use standard ServiceNow capabilities before custom scripting.
Missing security design	High	Define roles and ACL expectations before exposing tables.
Hardcoded configuration	Medium	Keep fare and business configuration in maintainable records.

21. Phase 2 Exit Criteria

- Approved data architecture is implemented.
- Required backend tables are created with correct names and scope.
- Required fields are created with appropriate data types.
- Relationships and reference fields are configured as designed.
- Forms/lists expose the required fields.
- Mandatory/choice/reference behavior is verified.
- Initial security/access expectations are validated.
- Test records can be created and retrieved successfully.
- Backend is ready for business rules, automation and ticket-generation logic.

22. Final Outcome

Phase 2 transforms the Phase 1 blueprint into the ServiceNow backend foundation. Correct table and field design is critical because later booking logic, fare calculation, workflows/flows, notifications, ACLs and ticket generation will depend on this data structure.

Phase 2 Deliverable: A clean, validated and maintainable ServiceNow data layer ready for the next implementation phase.

23. ServiceNow Implementation Evidence

The following screenshots are added to the Phase 2 document as direct implementation evidence for the Metro Station Details backend configuration. They complement the documented table-creation, field-creation and record-validation activities.

Metro Station Details — Table Configuration

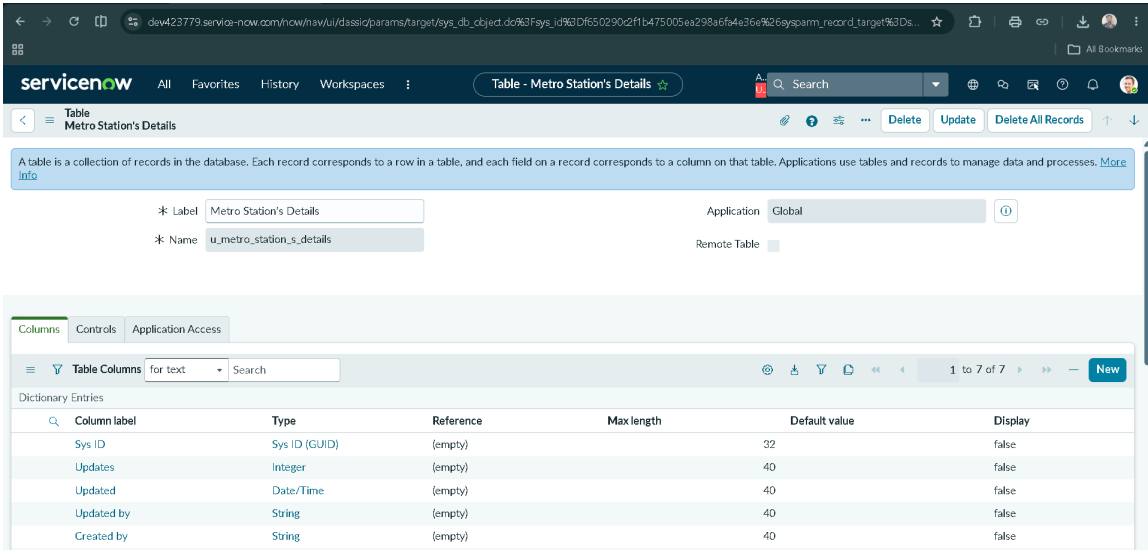


Figure 23. Metro Station Details table configuration showing the table definition and configured columns.

This evidence corresponds to the Phase 2 backend activity in which the Metro Station Details table is created as the master-data container for metro stations.

24. Station Record Creation and Verification

New Station Record

The screenshot shows the 'New Station Record' form in the servicenow application. The form is titled 'Metro Station Details - New Record' and includes a 'Station Name' field for data entry. The interface features a dark blue header with the servicenow logo and navigation links. A 'Submit' button is present at the bottom left of the form area.

Figure 24. Metro Station Details — New Record form showing the Station Name field used for station master-data creation.

Station Records List

	Station Name
☐	Kukatpally
☐	Jubilee Hills
☐	Uppal Stadium
☐	Ameerpet
☒	Panjagutta
☐	Madhapur
☐	L B Nagar

Figure 25. Metro Station Details list showing the configured station records, including Uppal Stadium, Panjagutta, Madhapur, L B Nagar, Kukatpally, Jubilee Hills and Ameerpet.

Together, these screens show the practical backend flow: the Station Name field is presented for record creation, and the saved station records can then be viewed in the Metro Station Details list. This supports the Phase 2 testing objective that valid station records can be created and retrieved successfully.